
LogicWard

Live OT Drift-Detection Appliance

Detect unauthorized PLC logic changes on a thermal power plant before they become operational risk.

Document: Design & Engineering Specification

Version: 1.1 (as built)

Date: 2026-07-28

Status: Built & verified — 98/98 automated checks across 7 suites

Audience: Engineering team & collaborating agents

At a glance

- A Raspberry Pi is the PLC (Modbus TCP) for a simulated thermal power plant.
- A second machine is the attacker; a laptop runs the detection engine + SOC dashboard.
- One event contract; three detection planes (cyber, physical, resource) on one bus.
- Rungs are *data* to baseline and diff — **no ladder-logic interpreter**.
- Reuses three existing repos: OT_SECURITY, PiSentinel, and Prahari.

Built for a cybersecurity hackathon. Reuses and re-themes prior work: OT_SECURITY (Modbus server + attacker), PiSentinel (dashboard + network/resource sensors), Prahari (scoring, demo choreography, SOC design language).

Contents

1	Executive summary	2
1.1	What a judge sees in 8 minutes	2
2	Implementation update (v1.1 — as built & verified)	2
2.1	Changed from the v1.0 draft	2
2.2	Verification (98/98) and how to run	3
3	Problem statement & scope	3
3.1	The problem	3
3.2	In scope	3
4	System architecture	4
4.1	Split deployment topology	4
4.2	Responsibility split	4
4.3	Data & control flows	4
5	The event bus — the spine (build this first)	5
5.1	The Event contract	5
5.2	Event type namespaces	6
5.3	Severity model (explainable, Prahari-derived)	6
5.4	Bus internals	7
5.5	Agent → engine forwarding protocol	7
5.6	Program channel (the structural surface)	7
5.7	Dashboard consumption	8
6	The L5X logic model — representing and diffing a “program”	8
6.1	The program is a real Rockwell L5X	8
6.2	The two detection surfaces	8
6.3	The six named mutation classes	9
6.4	<code>rung_to_register.py</code> — the cross-check map	9
7	Detection planes	9
7.1	Cyber plane (<code>engine/</code> , laptop) — the core	9
7.2	Physical plane (Pi agent) — three independent sensors	9
7.3	Resource plane (Pi agent) — bonus	9
8	Cross-cutting concerns	10
8.1	Evidence log & PDF forensic report (<code>dashboard/evidence.py</code>)	10
8.2	MITRE ATT&CK for ICS mapping (<code>engine/mitre_map.py</code>)	10
8.3	RBAC (<code>dashboard/app.py</code>)	10
8.4	Simulated mitigation (<code>engine/response.py</code>)	10
9	Dashboard (Flask, laptop)	11
10	Attacker (2nd machine)	11
10.1	<code>attacker/attacks.py</code>	11
10.2	<code>attacker/demo_sequence.py</code>	11
11	Repository layout	11
12	Reuse map — what is lifted from where	13
13	Locked design decisions	13

14 Build plan — delivered & verified	14
Appendix A — Glossary	14

1 Executive summary

LogicWard is a live Operational-Technology (OT) security appliance that continuously verifies a Programmable Logic Controller (PLC) against a **cryptographically hashed, approved baseline** and raises severity-ranked, MITRE-mapped alerts the instant the running logic or process state drifts from that baseline.

The demonstration models a **thermal power plant**. A **Raspberry Pi** acts as the PLC, running a Modbus TCP server plus a thin edge sensor agent. A **second machine** acts as the attacker. A **laptop** runs the detection engine and a professional Flask dashboard with role-based access control.

Core insight. A PLC “program” is *data*, not something we execute. LogicWard does *not* build a ladder-logic interpreter. It treats the approved rung set as structured data to baseline, and treats the PLC’s live Modbus registers as ground-truth reality. Detection is a continuous **diff** of live-vs-baseline across two surfaces: the structural program and the live register state.

Everything speaks **one event contract**. Three detection planes (cyber, physical, resource) all emit the same **Event** object onto a single **event bus**, which enriches each event with attacker identity and a MITRE ATT&CK for ICS technique, appends it to a tamper-evident evidence log, and streams it to the dashboard.

1.1 What a judge sees in 8 minutes

1. A quiet, professional SOC dashboard showing a live thermal plant running normally.
2. An attacker on a second machine drifts the PLC — changing a safety setpoint, inverting trip logic, injecting a rogue rung, hijacking an output coil.
3. LogicWard lights up in real time: a side-by-side baseline-vs-current diff with the exact changes highlighted, a severity-ranked alert feed, each alert mapped to a named MITRE ATT&CK for ICS technique.
4. Physical-tampering and resource-exhaustion (DDoS) attacks are caught by the same fabric.
5. One click exports a signed PDF forensic report: who, when, what changed, and the ICS technique used.

2 Implementation update (v1.1 — as built & verified)

The system is **built and passing 98 automated checks across 7 test suites**. Three v1.0 decisions were upgraded; this section is the authoritative delta — read it alongside the original sections that follow.

2.1 Changed from the v1.0 draft

- **Program format is real Rockwell L5X XML, not JSON.** The program is a genuine .L5X export; the engine parses it with `lxml`, **canonicalizes** it (strips volatile `ExportDate/EditedDate/revisions/CRC` and non-logic comments) and diffs canonical rung structure. A date-only re-export or a comment edit yields an *identical* structural hash (no false positives); an operator flip / setpoint drift / coil hijack changes it. Neutral-text ladder is parsed into instruction/operand structs so all six mutations are still named.
- **Baseline lock is HMAC-SHA256 signed** (`engine/baseline.py`); editing the locked baseline on disk breaks the signature. Integrity, not access control.

- **Passive FIM via watchdog** watches **both** files: the Pi's running `live.L5X` (out-of-band tamper → `cyber.program_file_modified`) and the laptop's signed baseline (change → HMAC re-verify → `cyber.baseline_tamper` critical, or `cyber.baseline_relocked` info).
- **GitHub-style side-by-side diff** (`engine/l5x_diff.py`, `difflib`): the dashboard highlights baseline-vs-live rungs with row- and word-level red/green.
- **SIEM-ready JSONL** — already satisfied by the evidence log.
- **Docker: deliberately not included** (hardware-coupled Pi sensors + live-demo robustness).

2.2 Verification (98/98) and how to run

Suite	Checks	Covers
<code>smoke_bus</code>	15/15	event bus, ingest/poll, forwarder, MITRE, evidence
<code>smoke_l5x</code>	16/16	L5X parse + canonicalize; no-false-positive; real changes
<code>smoke_plant</code>	14/14	thermal Modbus reads; setpoint write fires a trip; program endpoints
<code>smoke_drift</code>	18/18	HMAC lock/tamper; all 6 mutations w/ severity+MITRE+channel
<code>smoke_agent</code>	10/10	sensors (sim) + real watchdog FIM + agent→ingest wiring
<code>smoke_dashboard</code>	15/15	login/RBAC, drift surfacing, diff API, PDF, actions
<code>smoke_attacker</code>	10/10	every attack drives the expected detection end-to-end

Run: `dashboard python -m logicward.dashboard.app` (login `soc/soc123`); one-command live show `python -m logicward.attacker.demo_sequence` (brings up the whole stack, runs normal → attack → detection). Single-machine by default (embedded plant); `LOGICWARD_EMBED_PLANT=0` for the real Pi split.

3 Problem statement & scope

3.1 The problem

In OT/ICS environments, an attacker (or malicious insider) who can reach a PLC can silently alter its control logic or setpoints. Because Modbus TCP is unauthenticated and PLCs rarely log write operations, such a change can persist undetected until it causes a physical incident — a boiler over-pressure, a turbine overspeed, a safety interlock that no longer trips. **The problem: detect unauthorized PLC logic changes before they become operational risk.**

3.2 In scope

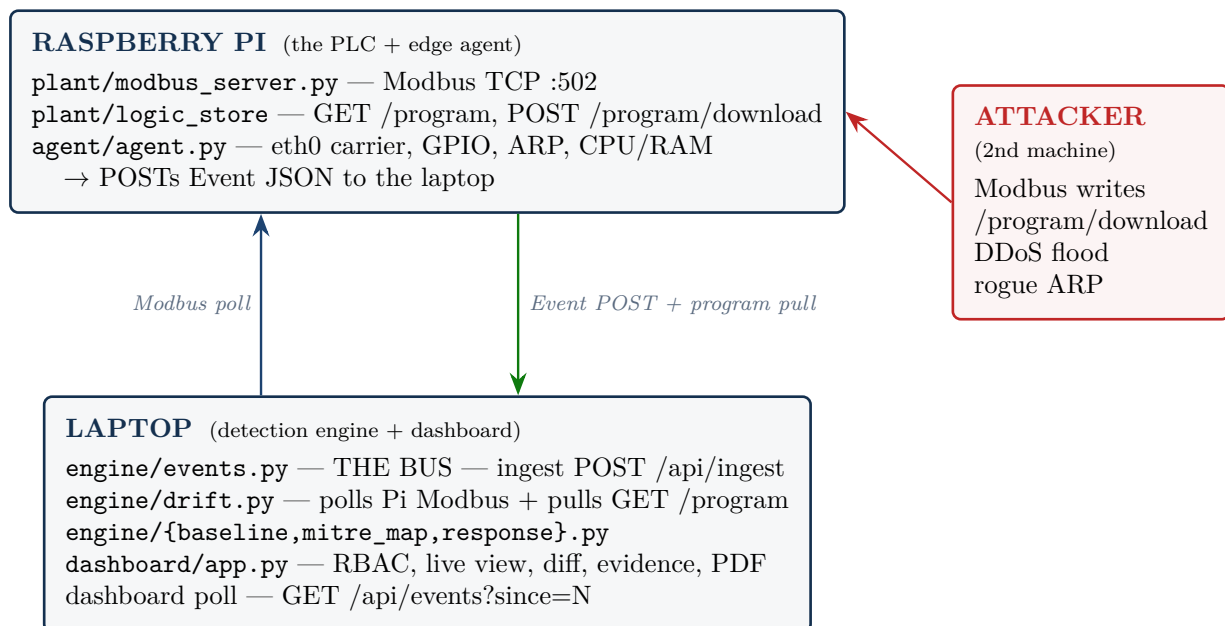
- A vulnerable-by-design Modbus TCP PLC simulating a thermal power plant.
- A structured representation of the PLC “program” (ladder rungs as JSON data).
- A hashed baseline of the approved program + register state.
- A structural + register **drift engine** detecting six named mutation classes.
- Three detection planes (cyber / physical / resource) unified on one event bus.
- Attacker tooling performing each named mutation, plus DDoS and rogue-device.
- A professional Flask dashboard: RBAC, live plant view, baseline-vs-current visual diff, severity-ranked alert feed, evidence log.
- MITRE ATT&CK *for ICS* mapping (rule-based, explainable).
- Exportable PDF forensic report and a scripted demo sequence.

Explicitly out of scope (anti-over-engineering guardrail). No ladder-logic execution engine. Rungs are data to baseline, mutate, and diff; the Pi's Modbus registers provide the live reality. Building an interpreter that *runs* the rungs is the single biggest trap and is deliberately excluded. Also excluded: ML black boxes in the detection path (detection is rule-based and explainable), and any production identity provider (RBAC is a demo-grade session/role gate).

4 System architecture

4.1 Split deployment topology

LogicWard runs across three hosts. The attacker only ever touches the Pi — mirroring reality: an adversary attacks the PLC, and the monitor observes it remotely.



4.2 Responsibility split

Host	Runs	Why here
Raspberry Pi	Modbus server (PLC), logic-store program endpoints, edge sensor agent	The agent must read local hardware — <code>/sys/class/net/eth0/carrier</code> , GPIO, ARP, CPU/RAM — so it lives next to the hardware. The PLC is the thing being attacked.
Laptop	Event bus, ingest endpoint, drift engine, baseline, MITRE map, response, evidence log, Flask dashboard	All analytical work and the UI. Polls the Pi's Modbus remotely; receives physical events from the agent.
Attacker	<code>attacks.py</code> , <code>demo_sequence.py</code>	Isolated adversary; reaches the Pi over the LAN.

4.3 Data & control flows

- Register truth (laptop ← Pi):** `drift.py` polls the Pi's Modbus registers/coils and compares to the baseline snapshot → `cyber.setpoint_drift`, `cyber.register_change`.

2. **Program truth (laptop ← Pi):** drift.py pulls GET /program and diffs it structurally against the baseline rung set → cyber.logic_inversion, cyber.condition_stripping, cyber.coil_hijack, cyber.rung_injection.
3. **Physical/resource truth (laptop ← Pi agent):** the agent samples hardware locally and POSTs physical.* and resource.* events to /api/ingest.
4. **Attacker → Pi:** register/coil mutations over Modbus; structural mutations via POST /program/download; DDoS via Modbus flood; rogue device by appearing on the LAN.
5. **Bus → everywhere:** every event, wherever born, lands on the single bus which enriches (identity + MITRE), logs (evidence), and serves (dashboard poll).

5 The event bus — the spine (build this first)

Everything depends on the bus, so it is implemented and verified before any detector.

5.1 The Event contract

One object, every plane, every host:

```
{
  "event_id": "3f9c1b7e-... (uuid4)",
  "type": "cyber.setpoint_drift",
  "timestamp": "2026-07-27T18:22:04.517Z",
  "source": "drift_engine",
  "severity": "high",
  "details": {
    "rung_id": "R07_DRUM_LOW_TRIP",
    "field": "threshold",
    "baseline": 220,
    "current": 40,
    "safety_critical": true
  }
}
```

Core fields (set by the emitter):

Field	Type	Meaning
event_id	string (uuid4)	Stable unique id — evidence, dedup, dashboard keys.
type	string	Namespaced event type (§4.2).
timestamp	string	ISO-8601 UTC, millisecond precision.
source	string	Detector/sensor that emitted it.
severity	enum	info low medium high critical.
details	object	Type-specific payload; enough to render a human explanation.

Bus-enriched fields (added on arrival, never by the emitter):

Field	Type	Meaning
identity	object	{who, mac, channel} — best-known attacker identity and the channel used (modbus-write, program-download, network, physical).
mitre	object	{tactic, technique_id, technique_name} — ATT&CK for ICS.
received_at	string	ISO-8601 UTC when the bus accepted it (may differ for buffered events).
seq	integer	Monotonic bus sequence number — the cursor the dashboard polls against.

5.2 Event type namespaces

Namespace	Emitted by	Examples
cyber.*	drift engine (laptop)	setpoint_drift, register_change, logic_inversion, condition_stripping, coil_hijack, rung_injection
physical.*	agent (Pi)	link_down, link_up, rogue_device, enclosure_open
resource.*	agent (Pi)	cpu_spike, mem_spike
response.*	response engine (laptop)	quarantine_device, operator_ack, recommend_safe_state, restore_baseline

5.3 Severity model (explainable, Prahari-derived)

Severity is computed, not hand-set, so it is defensible. Each event type has a **base weight**; `safety_critical` context multiplies it. The score maps to the enum band.

```
score = base_weight(type) * (SAFETY_MULTIPLIER if safety_critical else 1.0)

band:  score >= 85 -> critical    score >= 40 -> medium
       score >= 65 -> high       score >= 20 -> low
       else           -> info
```

Indicative base weights (all in one tunable table):

Event type	Base	Rationale
cyber.condition_stripping	80	Removing a safety interlock is the most dangerous single change.
cyber.logic_inversion	75	A trip that now fires backwards.
cyber.coil_hijack	70	Control redirected to the wrong actuator.
cyber.rung_injection	70	Foreign logic added to the program.
cyber.setpoint_drift	55	Dangerous on a safety rung; moderate otherwise.
cyber.register_change	35	Could be process noise or an attack; corroborated by other signals.
physical.enclosure_open	60	Physical access to the cabinet.
physical.rogue_device	50	Unknown MAC on the OT segment.
physical.link_down	45	Cable pull / network isolation.
resource.cpu_spike / mem_spike	40	Consistent with DDoS impact.

`SAFETY_MULTIPLIER` ≈ 1.25 , capped so score never exceeds 100. All weights live in a single constants block for judge-time tuning.

5.4 Bus internals

`engine/events.py` implements a thread-safe in-process publish/subscribe bus (a deque history + subscriber callbacks — the collector pattern proven in PiSentinel). On every `emit(event)`:

1. **Validate** the core schema; reject malformed events.
2. **Enrich** — assign `seq`, `received_at`; call `mitre_map.map(event)`; attach best-known identity.
3. **Score** — if severity was not set, compute it (§4.3).
4. **Sink 1 — Evidence log:** append to an append-only store (JSONL, upgradeable to SQLite).
5. **Sink 2 — Dashboard buffer:** append to the bounded history deque the poll endpoint reads.
6. **Sink 3 — Subscribers:** invoke live subscribers (e.g. auto-mitigation hooks).

Two emit paths converge on the same `emit()`: **local** (`drift.py`, `response.py` call it directly) and **remote** (the agent POSTs to `/api/ingest`; the handler validates token + schema, then calls `bus.emit()` per event).

5.5 Agent → engine forwarding protocol

The physical/resource sensors run on the Pi but their events must reach the bus on the laptop. The wire format **is** the Event object.

```
POST http://<laptop-ip>:8080/api/ingest
Headers:
  Content-Type: application/json
  X-LogicWard-Token: <shared secret from config.py>
Body:
  { "events": [ <Event>, <Event>, ... ] }      # batch of 1 or more

Responses:
  200 { "accepted": N }
  401 invalid or missing token
  422 schema validation failed { "errors": [...] }
```

Reliability. The agent buffers events in a local deque, flushes on a short interval or immediately on emit, and **retries with exponential backoff**, keeping the buffer if the laptop is briefly unreachable — so physical events are never silently lost. `event_id` makes ingest idempotent (retried duplicates are dropped by id).

Security. The shared token stops any other host on the LAN from injecting forged events. It is a config value, not a secret-management system — appropriate for the demo and documented as such.

5.6 Program channel (the structural surface)

The Pi exposes the PLC “program” — a real Rockwell **L5X** — over HTTP, mirroring how an engineering workstation reads/writes a PLC:

```
GET /program -> { "l5x": "<RSLogix5000Content...>", "hash": "sha256:...",
                  "controller": ..., "rung_count": ... }
POST /program/download -> replaces the running L5X (the attacker's structural channel
)
  body: { "l5x": "<...L5X XML...>" } (in a real PLC, this is a program download)
```

The laptop `drift.py` pulls `GET /program` each cycle, parses + canonicalizes the L5X, and diffs vs the baseline. The attacker performs structural mutations by `POST /program/download`; the running program persists to `live.L5X` so the FIM sensor also catches out-of-band file tampering. Setpoints/registers need no special channel — the attacker writes them over Modbus, the engine reads them over Modbus.

5.7 Dashboard consumption

```
GET /api/events?since=<seq> -> { "events": [ ...seq > since... ], "cursor": <latest seq> }
```

A `since-cursor` poll (PiSentinel’s proven pattern) rather than WebSockets — deliberately chosen for demo robustness (no socket reconnect fragility on stage). Poll cadence ~ 1 s gives a live feel.

6 The L5X logic model — representing and diffing a “program”

This is the heart of the cyber plane and the answer to *“how does an attacker change logic, and how do we see it, without executing anything?”*

6.1 The program is a real Rockwell L5X

The approved program is a genuine Studio 5000 export, `plant/program/ThermalPlant_baseline.L5X`. The engine parses it with `lxml` and **canonicalizes** it — stripping volatile export/edit timestamps, revisions, CRCs, and free-text comments — so a re-export with no logic change yields an *identical* structural hash (no false positives). Rungs are **data**, parsed and compared, **never executed**.

```
<Rung Number="0" Type="N">
  <Comment><![CDATA[SIL-2 Boiler drum level low-low -> trip feedwater]]></Comment>
  <Text><![CDATA[XIC(Plant_Running)[LES(Drum_Level,Drum_Level_LL_SP)]OTE(Feedwater_Trip);]]></Text>
</Rung>
```

parses to input contact `XIC(Plant_Running)`, compare `LES(Drum_Level,Drum_Level_LL_SP)`, output coil `OTE(Feedwater_Trip)`. `safety_critical` is derived from the output-coil name (a `*_Trip` coil is safety-critical). Setpoints (`*_SP` tags) carry values in the L5X and are mirrored into Modbus holding registers.

6.2 The two detection surfaces

Surface	Lives in	Mutation observed by
Setpoint values (<code>*_SP</code> tags)	L5X tag values, mirrored into Modbus HRs	Register diff (Modbus) <i>and</i> structural diff (L5X tag value)
operator/instructions, inputs, output_coil, rung existence	The L5X program (<code>GET /program</code>)	Structural diff — canonical L5X rung comparison vs baseline

The engine canonicalizes and diffs the L5X program (structural mutations) and independently diffs live Modbus registers/coils (setpoint writes + unauthorized commands). A setpoint change is corroborated on both surfaces; volatile noise on neither. Both detection paths have real teeth.

6.3 The six named mutation classes

Mutation	What the attacker changes	Detected as	Severity
Setpoint drift	A *_SP setpoint value (Modbus HR + mirrored L5X tag)	setpoint_ drift	high (safety)
Logic inversion	Compare op flipped (LES↔GRT) or contact toggled (XIC↔XIO)	logic_ inversion	high
Condition stripping	A safety input instruction removed from a rung	condition_ stripping	critical
Coil hijack	A rung's output coil (OTE/OTL/OTU) repointed	coil_hijack	high
Rung injection	A new, unapproved rung added	rung_ injection	high
Raw register change	A holding register or control coil forced over Modbus	register_ change	medium

6.4 rung_to_register.py — the cross-check map

Maps each rung's contacts/coils/thresholds to concrete Modbus addresses. This lets the engine know which live register backs each setpoint, and cross-check that a rung's declared output coil corresponds to real coil behaviour — catching a coil hijack even before the physical effect manifests.

7 Detection planes

7.1 Cyber plane (engine/, laptop) — the core

- **baseline.py** — captures the approved state (L5X program, structural hash, setpoints, register/coil snapshot) and signs the manifest with **HMAC-SHA256**; editing the locked baseline on disk breaks the signature. Re-baselining is a privileged action, logged as evidence.
- **drift.py** — the diff loop. Each cycle: pull GET /program; poll Modbus; run the canonical L5X structural diff and the register diff; emit one event per mutation with a precise details payload (rung id, field, baseline value, current value, **safety_critical**). Idempotent — a persistent mutation is reported once (by content hash), with an “ongoing” flag.
- **Severity** by criticality: safety-critical rungs escalate (§4.3).

7.2 Physical plane (Pi agent) — three independent sensors

- **link_watch.py** — reads /sys/class/net/eth0/carrier; 0 = cable pulled → `physical.link_down` (and `link_up` on restore).
- **arp_watch.py** — reuses PiSentinel ARP discovery + an **allowlist**; any MAC not on the allowlist → `physical.rogue_device` with vendor lookup. The allowlist is captured alongside the baseline.
- **gpio_tamper.py** — enclosure-open switch via `gpiozero`, **with a software-simulation fallback** so the pipeline works with no hardware (a simulated toggle emits the identical `physical.enclosure_open` event). Keeps the demo runnable on any laptop.

7.3 Resource plane (Pi agent) — bonus

`resource.py` reuses PiSentinel's `psutil` / `/proc` sampling for CPU/RAM. Sustained spikes → `resource.cpu_spike` / `resource.mem_spike`, demonstrating the operational impact of a DDoS flood against the Modbus server.

8 Cross-cutting concerns

8.1 Evidence log & PDF forensic report (dashboard/evidence.py)

Every event records **who** (source identity — attacker IP/MAC/channel), **when** (timestamp), and **what** (drift details). The log is append-only. A SOC Analyst exports a **PDF forensic report**: an incident header, a chronological event table, the baseline-vs-current diffs per cyber event, and the MITRE technique per event — the artifact a real SOC hands an investigator.

8.2 MITRE ATT&CK for ICS mapping (engine/mitre_map.py)

A **rule-based, explainable** table maps each event type to a technique in the **ATT&CK for ICS** matrix (the OT matrix, *not* enterprise ATT&CK). No ML — a judge can read the mapping. Technique IDs, names, and tactics were **verified 2026-07-28 against the live matrix** (attack.mitre.org/matrices/ics).

Event type	Tactic (ICS)	Technique	ID
setpoint_drift	Impair Process Control	Modify Parameter	T0836
logic_inversion / condition_stripping / coil_hijack	Persistence	Modify Program	T0889
rung_injection	Lateral Movement	Program Down-load	T0843
register_change	Impair Process Control	Unauthorized Command Message	T0855
program_file_modified	Persistence	Modify Program	T0889
physical.rogue_device	Initial Access	Rogue Master	T0848
physical.link_down / resource.cpu_spike	Inhibit Response Function	Denial of Service	T0814
physical.enclosure_open	Initial Access	<i>no direct ICS technique</i>	N/A

Each mapping carries a `verified: true` flag. The two honest N/A rows (physical enclosure tamper; baseline-integrity tamper) have no direct ATT&CK-for-ICS technique and are deliberately **not** assigned a fabricated ID.

8.3 RBAC (dashboard/app.py)

Three roles, session-based, enforced server-side.

Role	Can see	Can do
Operator	Live plant view, alert feed	Acknowledge alerts (<code>operator_ack</code>)
Engineer	+ baseline & diff detail	Capture/re-baseline, restore baseline
SOC Analyst	Everything	Evidence log, PDF export, run demo, all mitigation actions

8.4 Simulated mitigation (engine/response.py)

Per the chosen posture — **detect and simulate mitigation** — the dashboard offers non-destructive response actions, each emitting a `response.*` event logged as evidence: **quarantine rogue device** (simulated segment isolation), **recommend safe-state** (surfaces the recommended safe action on a

safety-critical drift; does not command the PLC), **restore baseline** (Engineer pushes the approved program back), **operator acknowledge** (human-in-the-loop clear).

9 Dashboard (Flask, laptop)

Built on PiSentinel’s server-rendered Jinja shell dressed with Prahari’s SOC design language (navy sidebar, graded severity palette, badge tokens).

View	Contents
Overview	Live plant status, impact metrics strip, severity-ranked alert feed, LIVE indicator.
Live Plant	Thermal-plant mimic: boiler drum level, steam temp/pressure, turbine speed, generator MW, feedwater pumps, trip coils — driven by live Modbus reads.
Baseline vs Current	Side-by-side rung + register diff, changed fields highlighted, per-change severity + MITRE technique.
Alerts	Full severity/plane-coded feed, flashes on arrival.
Evidence	The forensic log + one-click PDF export.
Demo (SOC Analyst)	Buttons to run the scripted scenarios.

Live data via GET /api/events?since=N + GET /api/plant (current register snapshot) polled ~1s.

10 Attacker (2nd machine)

10.1 attacker/attacks.py

Built on OT_SECURITY’s modisy.py raw Modbus client.

Attack	Mechanism	Triggers
Setpoint drift	Modbus FC06 write to a threshold HR	cyber.setpoint_drift
Logic inversion	POST /program/download (flipped operator)	cyber.logic_inversion
Condition stripping	POST /program/download (removed safety input)	cyber.condition_stripping
Coil hijack	POST /program/download (repointed coil)	cyber.coil_hijack
Rung injection	POST /program/download (added rung)	cyber.rung_injection
Raw register change	Modbus FC06/FC16 write	cyber.register_change
DDoS	Modbus request flood	resource.cpu_spike
Rogue device	Appear on the LAN (new MAC)	physical.rogue_device

10.2 attacker/demo_sequence.py

Reuses Prahari’s timed choreography: **normal ops** → **attack** → **detection**, driving the attacks in a scripted order with pauses so a live audience can follow each detection as it lands. Ordered so severities build to a climax (safety-critical condition-strip last).

11 Repository layout

```

logicward/
+-- plant/                                # -- on the Pi --
|   +-- modbus_server.py                  # OT_SECURITY server, re-themed to a thermal plant
|   +-- program/ThermalPlant_baseline.L5X # approved program (real Rockwell L5X) --
|   DATA, never executed
|   +-- logic_store.py                    # GET /program + POST /program/download; persists live.
|   L5X
|   \-- rung_to_register.py               # L5X tag <-> Modbus address map
+-- agent/                                # -- on the Pi --
|   +-- forwarder.py                     # buffered/retrying HTTP POST client to /api/ingest
|   +-- agent.py                          # runs sensors locally; forwards Events to the laptop
|   \-- sensors/
|       +-- link_watch.py                 # /sys/class/net/eth0/carrier (+ sim)
|       +-- arp_watch.py                  # PiSentinel ARP allowlist diff
|       +-- gpio_tamper.py                 # gpiozero + software-sim fallback
|       +-- resource.py                   # PiSentinel CPU/RAM sampling
|       \-- fim_watch.py                  # watchdog FIM: live.L5X + signed baseline
+-- engine/                               # -- on the laptop --
|   +-- events.py                         # * THE BUS + severity + evidence sink
|   +-- server.py                         # /api/ingest + /api/events blueprint
|   +-- l5x.py / l5x_diff.py              # L5X parse+canonicalize+hash; GitHub-style diff
|   +-- baseline.py                       # HMAC-SHA256 signed baseline lock
|   +-- drift.py                          # canonical L5X diff + register diff (6 mutations)
|   +-- mitre_map.py                      # event type -> ATT&CK for ICS (verified)
|   +-- response.py                       # simulated mitigation actions
|   \-- sources.py / modbus_client.py     # EmbeddedPlant/RemotePlant; laptop reads the Pi
+-- dashboard/                            # -- on the laptop --
|   +-- app.py                            # Flask + RBAC; mounts ingest/poll; drift loop; baseline
|   FIM
|   +-- evidence.py                       # who/when/what log -> reportlab PDF forensic export
|   +-- templates/ + static/              # login + SOC dashboard (live view, red/green diff)
+-- attacker/                             # -- on the 2nd machine --
|   +-- attacks.py                        # the 6 mutations + DDoS + rogue device
|   \-- demo_sequence.py                  # scripted normal -> attack -> detection (self-contained)
+-- tests/                                # smoke_{bus,l5x,plant,drift,agent,dashboard,attacker}.py
+-- config.py                             # Pi IP, ingest URL, token, HMAC key, poll intervals
+-- requirements.txt
\-- README.md

```

12 Reuse map — what is lifted from where

Source	Lifted asset	Becomes
OT_SECU-RITY	plc_simulator.py raw Modbus TCP server (MBAP, FC 01–11/2B, threaded sim)	plant/modbus_server.py (re-themed thermal)
OT_SECU-RITY	modisy.py raw client (scan/dump/write/estop/flood/replay)	attacker/attacks.py
PiSentinel	app.py Flask polling shell + collector pattern	dashboard/app.py, bus poll model
PiSentinel	network_trace.py ARP discovery + known/new tagging	arp_watch.py
PiSentinel	psutil//proc system stats	resource.py
PiSentinel	style.css + Jinja templates	dashboard shell
Prahari	score.py / rules.py scoring philosophy	severity model in events.py
Prahari	attack.py + DEMO_SCRIPT.md choreography	demo_sequence.py
Prahari	auth (sessions + roles)	dashboard/app.py RBAC
Prahari	index.css design tokens	dashboard styling

13 Locked design decisions

1. **Logic model = real Rockwell L5X.** A genuine .L5X export, parsed + canonicalized (volatile dates/revisions/CRC/comments stripped) and diffed structurally; setpoints mirror into Modbus registers for corroboration. Rungs are data — no interpreter.
2. **Baseline = HMAC-SHA256 signed lock** — tamper breaks the signature; a passive watchdog FIM watches both the live program and the signed baseline.
3. **RBAC = Operator / Engineer / SOC Analyst.**
4. **Response = detect + simulate mitigation** (non-destructive, each logged as evidence).
5. **Split architecture.** Pi = Modbus PLC + edge sensor agent + L5X program endpoints. Laptop = engine + dashboard; polls Modbus remotely, ingests physical events over HTTP.
6. **Transport = HTTP.** Agent→engine = POST /api/ingest (batched Events, token auth, retry/buffer). Dashboard = GET /api/events?since=N polling.
7. **MITRE = ATT&CK for ICS**, rule-based, technique IDs verified against the live matrix.
8. **Docker deliberately not included** (hardware-coupled Pi sensors + live-demo robustness).

14 Build plan — delivered & verified

Stage	Deliverable	Status
0-1	This document (v1.1) + scaffold + <code>config.py</code>	delivered
2	<code>engine/events.py</code> + <code>server.py</code> — bus, ingest, poll, evidence	smoke_bus 15/15
3	<code>agent/</code> forwarder + sensors (sim fallbacks) + <code>fim_watch</code>	smoke_agent 10/10
4	<code>plant/</code> Modbus + <code>ThermalPlant_baseline.L5X</code> + program endpoints; <code>engine/15x</code>	smoke_plant 14/14, smoke_15x 16/16
5	<code>engine/baseline.py</code> (HMAC) + <code>engine/drift.py</code>	smoke_drift 18/18 (6 mutations)
6	<code>engine/mitre_map.py</code> (verified) + <code>response.py</code> + <code>15x_diff</code>	folded into suites
7	<code>dashboard/</code> (RBAC, live view, GitHub diff, feed, evidence, PDF)	smoke_dashboard 15/15
8	<code>attacker/attacks.py</code> + <code>demo_sequence.py</code>	smoke_attacker 10/10 + live demo
9	Two-host integration on real Pi + laptop	pending Pi setup

Delivered: 98/98 checks across 7 suites. Only stage 9 (real hardware) remains.

Appendix A — Glossary

PLC Programmable Logic Controller; the industrial computer running control logic.

OT / ICS Operational Technology / Industrial Control Systems.

Modbus TCP A common, unauthenticated industrial protocol. Coils (RW bits), discrete inputs (RO bits), input registers (RO 16-bit), holding registers (RW 16-bit).

Ladder logic / rung The graphical PLC programming model; a rung is one logical statement (inputs → operator → output coil).

Setpoint A configured threshold the control logic compares against.

Baseline The hashed, approved snapshot of program + register state that live reality is diffed against.

Drift Any deviation of live state from the baseline.

ATT&CK for ICS MITRE's adversary technique matrix specific to industrial control systems.